

発表スクリプト — IntSeqBERT @ CICM 2026

- 英語表現の目安: B2 下位。専門用語は残し、短い文と直接的な説明を使う。
- slides.tex の本編 19 枚・予備 4 枚と同じ順序。スライド本文の目安は B2 中位。
- > の段落と Bridge が読み上げ部分。日本語メモは読み上げない。
- 発表 25 分＋質疑 5 分を想定。下の時間は図の説明・間を含む練習用の配分であり、実測ではない。
- 本編の読み上げ語数と所要時間の目安は末尾参照。予備スライドは質疑用で、本編時間に含めない。

Slide 1: Title — 0:00–0:40

Good morning, everyone. I am Kazuhisa Nakasho, from Iwate Prefectural University in Japan. Today I will talk about IntSeqBERT. It is a Transformer encoder for integer sequences in the OEIS. We want to learn the arithmetic structure of these sequences. Our code and data are public on GitHub.

Bridge: “Let me start with a recent comment by Terence Tao.”

Slide 2: Why Finding Good Problems Matters — 0:40–1:50

In a recent Mastodon post, Terence Tao raised a concern about open problems. We may solve good problems faster than we find new ones. But how can this happen if we can ask infinitely many questions?

Tao gives a simple example. A country can be surrounded by an ocean and still lack drinking water. In mathematics, we can also have many questions but few useful ones. Good problems help us find new ideas and connections.

This motivates our long-term goal. Can AI help us find new mathematical connections and promising conjectures? Our work takes a first step by learning arithmetic patterns in integer sequences.

Bridge: “Let me show how numerical patterns can lead to new mathematics.”

発表メモ: スライド上の英文引用は Tao 氏の原文。口頭では B2 下位を目安に言い換える。投稿の問題提起と、そこから導く本研究の動機を分けて説明する。

出典: Terence Tao, Mastodon, post 1/4。ご提示の本文に基づく。投稿日は未確認のため記載しない。

Slide 3: Long-Term Goal — AI for Mathematical Discovery — 1:50–3:20

New connections have often led to important conjectures. McKay noticed that 196,884 is 196,883 plus one. The first number appears in the j -function. The second is a dimension related to the Monster group. This observation led to Monstrous Moonshine. The BSD conjecture and mirror symmetry also began with numerical patterns.

Today, AI systems can take part in mathematical discovery. Examples include AlphaProof, the Ramanujan Machine, and FunSearch. Our long-term goal is to find similar arithmetic patterns in sequences from different fields. These connections could help us make new conjectures.

We use about 392,000 OEIS sequences, each with a mathematical definition. To compare them, we need a

representation that a computer can process. In this paper, we build one and measure how much arithmetic structure it learns.

Bridge: “There is one main difficulty: learning multiplication.”

Slide 4: Why Multiplication Is Hard for Deep Learning — 3:20–5:10

Standard neural models have difficulty learning rules based on multiplication. One reason is called spectral bias. Networks tend to learn low-frequency patterns first. These are more like addition.

In theory, multiplication does not need many layers. A threshold circuit with three layers can multiply two n -bit numbers. But standard training does not easily find such a solution. In one study, GPT-4 answered 59 percent of three-digit multiplication questions correctly. For four-digit multiplication, the score was only 4 percent.

Even multiplication modulo a prime can need a very long training period. A model may learn the general rule only much later. This is called grokking. Another study used twelve layers and a special training method to multiply fifteen-digit numbers.

Tokenisation creates another problem. Values outside a fixed vocabulary all become the same unknown token, UNK. The token IDs also do not directly show arithmetic structure. This matters for the OEIS. Its values range from single digits to around ten to the power of 210.

Bridge: “Our idea is to give the model useful information from number theory.”

発表メモ: 「乗算は不可能」と断言せず、理論上の計算可能性と通常の学習の難しさを区別する。出典はスライドと予備 4 に表示。

Slide 5: Key Idea — Show Multiplicative Structure in the Input — 5:10–6:40

We use modular arithmetic. The residue of x modulo m is the remainder when we divide x by m . This operation is a ring homomorphism. This means that it preserves both addition and multiplication.

The residues keep information about multiplication, even when the original numbers are very large. For a number as large as ten to the power of 210, the residue is still between zero and m minus one.

Number theory also explains periodic patterns in these residues. Fermat’s little theorem describes how powers repeat modulo a prime. Quadratic reciprocity connects residue information across primes. The Chinese Remainder Theorem, or CRT, combines information from different prime powers.

This leads to our hypothesis. We give the network residues for many moduli. We call this the modulo spectrum. Then information about multiplication is available in the input layer, at depth zero. The model does not have to learn all this structure from the original values alone.

Bridge: “Now let me explain the task we use to test this idea.”

Slide 6: Task — Masked Sequence Modelling on OEIS — 6:40–8:00

Our task is masked sequence modelling. It is like BERT, but with integers. We take up to the first 128 terms of a sequence and hide 15 percent of the positions. The model predicts the hidden values from the other terms. This tests how well it has learned the rule behind the sequence. If we hide only the last term, the task becomes next-term prediction.

An earlier benchmark, FACT, tested a standard Transformer on five OEIS tasks. Predicting masked terms was the hardest task. Our Vanilla baseline uses that Transformer architecture.

For each hidden position, our model predicts three things. First, the magnitude, or size, on a log scale. Second, the sign: positive, negative, or zero. Third, the residues for every modulus from 2 to 101.

Bridge: “The model uses two input streams to make these predictions.”

Slide 7: IntSeqBERT — Dual-Stream Architecture — 8:00–9:40

This figure shows IntSeqBERT. The first stream is the magnitude stream. It represents the log-scale size and the sign using four numbers.

The second stream is the modulo stream. For each modulus from 2 to 101, we put the residue on a unit circle using sine and cosine. This gives us 200 numbers in total. It uses the idea of sinusoidal positional encoding, but here we encode residues instead of positions. On the circle, residue m minus one is close to residue zero. This matches the cyclic structure of modular arithmetic.

We combine the two streams with FiLM. The modulo stream gives a scale and a shift. We apply them to each element of the magnitude stream. The result goes into a standard pre-LN Transformer encoder. We test three model sizes. The largest has 91.5 million parameters.

Bridge: “We train this encoder with three prediction tasks.”

発表メモ: 図を左から右へ指す。FiLM の数式全体は読み上げず、scale と shift を説明する。

Slide 8: Training — Multi-Task Objective — 9:40–10:50

The encoder has three prediction heads: magnitude, sign, and modulo. The magnitude head also predicts uncertainty. The Solver will use this later. The modulo head has one classifier for each of the 100 moduli. We normalise each loss by its maximum entropy. This keeps the losses on a similar scale. We add the three losses, with a weight of two for the modulo loss.

We train on 219,765 sequences for 200 epochs. We use FP32 because very large values cause overflow problems with FP16. All experiments use one RTX 3070 Ti GPU with eight gigabytes of memory. Other researchers can repeat the experiments on similar hardware.

Bridge: “Next, we need to turn the model’s predictions into integers.”

Slide 9: Neural-Symbolic Decoding — the CRT Solver — 10:50–12:20

The network gives probability distributions for magnitude, sign, and residues. The Solver uses these distributions to produce integer candidates.

First, magnitude and uncertainty define a three-sigma search interval. For a small interval, we check every candidate. For a larger interval, we use beam search with the Chinese Remainder Theorem. We start with the moduli for which the model is most confident. For very large intervals, sparse CRT produces candidates directly. The three search modes and their limits are shown on the slide.

We call this a neural-symbolic system. The neural network makes predictions, and the Solver applies exact mathematical rules. We put residue arithmetic and CRT into the system ourselves. The network does not have to learn these rules from data. Is this a strength or a limitation? I will return to this question after the results.

Bridge: “Before the results, let us look at the data and the evaluation measures.”

Slide 10: Dataset — 274,705 OEIS Sequences — 12:20–13:30

We start with 391,710 OEIS entries and keep 274,705, or about 70 percent. We split them into training, validation, and test sets in a ratio of eight to one to one.

We exclude eleven tags in three groups. The first group contains sequences whose patterns depend on how values are written. Examples include decimal expansions and continued fractions. The second group does not fit our task setting. Tables are one example. The third group contains quality labels.

These choices are open to discussion. For example, the labels less and dumb depend on people’s judgement. Tables could also be converted into sequences. Future benchmarks may make different choices. For evaluation, we divide values into five size groups, which we call magnitude buckets. They range from Small, below 100, to Astronomical, at least ten to the power of fifty.

Bridge: “One measure needs a short explanation.”

Slide 11: Metrics — and Three Questions — 13:30–14:50

Magnitude accuracy, or Acc zero point five, does not require an exact integer match. It checks whether the prediction is within zero point five on the log scale. This is a factor of about 3.16. For example, if the true value is one million, predictions from about 320,000 to 3.2 million count as correct. We use this measure because the magnitude prediction helps set the Solver’s search range.

We measure exact integer matches separately, using Solver Top-k accuracy. MMA is the average classification accuracy across all 100 moduli. NIG means normalised information gain. We use it to measure learning for each modulus. Zero means no information gain over a uniform prediction. One means perfect prediction.

We ask three questions about the results. Does the dual stream help? Where does it help? And can we predict exact integers?

Bridge: “Let us start with the first question.”

Slide 12: Q1 — Does It Help? Yes, Consistently — 14:50–16:10

Yes, the dual stream helps. For the Large model, magnitude accuracy is 95.85 percent. This is 8.9 percentage points above the Vanilla baseline. Mean modulo accuracy is 50.38 percent, which is 4.5 points above Vanilla.

Now look at the ablation row. This is the model without the modulo stream. Removing that stream reduces mean modulo accuracy by 15.2 points. It also reduces magnitude accuracy by 6.2 points. So the modulo stream helps with both tasks. The residues limit which integer values are possible. This also helps the model predict magnitude.

We see the same pattern for the Small and Middle models. The backup slides show how the results change with model size.

Bridge: “The second question is where the model helps most.”

Slide 13: Q2 — Where? Exactly Where Tokenisation Breaks — 16:10–17:20

This plot compares predicted magnitude with true magnitude. Look at the Vanilla model. Above its vocabulary limit, all values become UNK, and its predictions become less accurate. In the Large bucket, its error is about thirteen times our model’s error. The overall R-squared values are 0.988 for IntSeqBERT and 0.943 for Vanilla.

We need to be careful with overall scores. Most OEIS sequences grow slowly, so easy cases have a large effect on these scores. That is why we also report results for each size group. The clearest gains appear in the Large bucket and above. Our magnitude predictions are still useful there, while the baseline has difficulty.

Bridge: “Now for the third question: can we predict the exact integer?”

Slide 14: Q3 — The Strictest Test: Exact Next-Term Prediction — 17:20–18:40

We test the full system on 10,000 sequences kept separate for testing. The task is to predict the exact last term. Vanilla gets 2.59 percent correct with its first candidate. IntSeqBERT gets 19.09 percent. That is about 7.4 times the baseline score. With ten candidates, our score rises to 26.23 percent.

For small values, our Top-1 score is 68.3 percent. For medium values, it is 20.8 percent. Vanilla scores zero beyond its vocabulary. But our model also scores close to zero for the Large bucket and above.

To reconstruct a large integer, CRT needs correct residues for many moduli at the same time. Our average modulo accuracy is about 50 percent, so getting enough residues correct is very difficult.

Bridge: “So, is 19 percent a success or a failure?”

Slide 15: What Does 19% Mean? — A Useful Negative Result — 18:40–20:00

There are two useful ways to understand this result. First, exact prediction is still hard, even when we give the model number-theoretic features. The score of 2.59 percent for Vanilla shows how difficult this task is for a tokenised Transformer. Our results measure a limit of end-to-end learning in this setting. This is a useful negative result: it tells us what the models still cannot do well.

Second, the results support combining neural networks with symbolic methods. The CRT solver adds exact rules that the network does not learn well on its own.

Earlier, I asked whether adding mathematical rules is a strength or a limitation. In this study, the features are also a tool for measurement. We can remove the modulo stream and measure the change. The drop of 15.2 points shows how much it helps the model learn arithmetic structure.

Bridge: “Accuracy is one way to evaluate the model. Next, let us look at what it learned about number theory.”

Slide 16: But What Did the Network Learn? The NIG Spectrum — 20:00–21:10

This plot shows NIG for every modulus from 2 to 101. IntSeqBERT scores above both baselines across the whole spectrum. But the scores are not the same for every modulus. The peaks follow a clear pattern.

Across all models and sizes, the best modulus is 96. This is two to the power of five, times three. The second best is 2, which tells us whether a number is even or odd. We call this parity. Parity patterns are common in the OEIS. For modulus 2, accuracy is 85.7 percent for IntSeqBERT, 81.4 for Vanilla, and 72.1 without the modulo stream.

Bridge: “Why does 96 work so well? The next plot helps explain this.”

Slide 17: Finding — Higher NIG with a Lower Totient Ratio — 21:10–22:40

Here we compare NIG with Euler’s totient ratio, $\phi(m)/m$. There is a strong negative correlation: minus 0.851. The p-value is below ten to the power of minus 28. In general, a lower totient ratio goes with a higher NIG score.

We explain this using CRT. If we know x modulo 96, we also know x modulo 2, 4, 8, 16, and 32, as well as modulo 3. So one modulus gives information about several smaller moduli. Composite moduli with many small prime factors combine this information.

This is an experimental result, not a theorem. It suggests that we should test highly composite numbers when we extend the spectrum beyond 101. These are numbers with more divisors than any smaller positive integer. They may be more useful than adding more primes.

The spectrum also describes the arithmetic patterns in a dataset. We could use these spectra to compare sequences or datasets.

Bridge: “Comparing sequences brings us back to our original goal.”

Slide 18: Positioning in Computer Mathematics — 22:40–23:50

How could this work help computer mathematics? Program synthesis methods, such as Alien Coding, QSynt, and Learning Conjecturing, find rules that generate sequences. Our method learns arithmetic patterns shared by many sequences. These two approaches could work together.

Program synthesis asks, what program generates this sequence? We ask, which sequences have similar number-theoretic structure? Their rules may look very different, but their arithmetic patterns may be similar.

Our learned representations could guide the search for a program. They could also help us find similar sequences in a large collection. These are possible future uses. Finding such connections could help us generate new conjectures. This is the long-term goal I described at the start.

Bridge: “Let me finish with four main points.”

Slide 19: Conclusion — 23:50–25:00

First, the modulo spectrum uses features based on number theory. A ring homomorphism makes information about multiplication available in the input, at depth zero.

Second, the model improves results at every size. Exact next-term prediction is about 7.4 times better than the baseline.

Third, composite moduli combine arithmetic information. We find higher NIG scores with lower totient ratios.

Fourth, exact prediction of OEIS terms is still hard. This supports further work on neural-symbolic methods.

Our next steps are to test approximate CRT for large integers, add highly composite moduli, and train larger models. We also want to use similar sequences to help generate conjectures. Our code and data are on GitHub. Thank you for listening.

Backup Scripts (質疑用・本編 25 分には含めない)

Backup 1: Full Results & Scaling Behaviour

The model ranking stays the same at all three sizes: IntSeqBERT, then Vanilla, then the ablation model. From Small to Large, mean modulo accuracy rises by ten percentage points. Magnitude accuracy rises by only 1.1 points. So model size helps modulo prediction more than magnitude prediction. Solver Top-1 accuracy rises from 14.05 to 19.09 percent. At epoch 200, the training loss is 1.00 and the validation loss is 1.01. These results show no clear sign of overfitting. We plan to train at a larger scale.

Backup 2: Why Not Digit-Based Tokenisation?

Another option is to use one token for each decimal digit. We have not compared this approach with our method in this paper. Our input gives the model residues directly. With digit tokens, the model would need to calculate the residues from the digits. The prediction task also changes. It would generate a sequence of digits instead of predicting one integer value. We plan to compare the two approaches under the same conditions.

Backup 3: Limitations

Large integers remain difficult because many residues must be correct at the same time. In 13.4 percent of Solver calls, no valid candidate is found. Approximate CRT or less strict residue conditions may help.

The dataset also affects the results. Many sequences contain only non-negative values, which helps sign accuracy. Many grow slowly, which helps overall magnitude accuracy.

We used one eight-gigabyte GPU and one random seed, 42. The baseline vocabulary has 20,003 tokens. Vanilla uses the same architecture as FACT, but with our training settings. We do not reproduce the numbers reported in the FACT paper.

Backup 4: How Hard Is Multiplication for Neural Networks?

The answer depends on the circuit type and the training method. In circuit theory, n -bit multiplication is not in AC zero, the class of constant-depth AND and OR circuits. But threshold circuits can do it with three layers and polynomial size. Fewer layers are not enough in that setting. Iterated products can be computed with four layers.

There is also a connection between Transformers and circuit complexity. The cited result gives uniform TC zero as an upper bound for log-precision Transformers.

The experiments on this slide show difficulties in learning multiplication. In the cited GPT-4 study, zero-shot accuracy falls from 59 percent for three-digit multiplication to 4 percent for four-digit multiplication. In the grokking study, a two-layer Transformer learns to generalise only after a very long training period. The final study reports failures with standard training even at 90 billion parameters. A twelve-layer model succeeds on

fifteen-digit products with special implicit chain-of-thought training. Its attention forms a binary tree of partial products.

発表メモ: 専門用語の $AC^0 \cdot TC^0$ 等は維持。引用文献の詳細は予備スライド 4 を示す。

練習メモ

- 冒頭は Tao 氏の問題提起→歴史的な数値観察→本研究の長期目標の順。新規スライド分は冒頭の説明を短くし、25 分の練習用配分を維持。
 - 本編は Slide 1~19。Slide 9 の「strength or limitation?」を Slide 15 で回収する。
 - 結果は Q1 → Q2 → Q3 →限界の解釈→ NIG →数論的な分析の順。旧稿の並べ替え提案は適用済み。
 - percentage points は割合の差、times は倍率として読み分ける。
 - modulus は単数、moduli は複数。初出の residue と parity は原稿内で説明する。
 - 図・数式・引用文献の細部は必要に応じて指し示す。全てを読み上げる必要はない。
 - 各スライドの時間配分は目標。通し練習で測り、25 分に収まるよう間と図の説明を調整する。
-
- 本編読み上げ語数: 約 2,530 語（本文と Bridge、数字・略語は表記上の 1 語として概算）。110 語/分なら約 23.0 分、100 語/分なら約 25.3 分。図の説明や間は別に必要なので、25 分枠では 110 語/分程度を目安に通し練習する。